



UNIDADE I

Programação Estruturada em C

Prof. Me. Fábio Assis

Introdução à Programação Estruturada em C

- C é uma linguagem de programação estruturada e compilada, padronizada pela ISO.
- Desenvolvida nos anos de 1970, a linguagem C tornou-se a base de muitas outras linguagens.
- Assim como a matemática é a base de diversas ciências, a linguagem C serve de alicerce para o desenvolvimento de muitas outras linguagens modernas, como C++, C# e Java.

Destques da linguagem C:

- Flexibilidade, portabilidade e eficiência.

Ampla utilização no desenvolvimento de diversas aplicações, como:

- Jogos eletrônicos;
- Sistemas de controle embarcados, como os de satélites;
- Sistemas operacionais, como Unix e Linux.

Estrutura de um Programa em C

A estrutura básica de um programa em C segue um padrão simples:

- A primeira parte de um programa em C geralmente inclui diretivas com o uso de #include, que são instruções processadas antes da compilação pelo pré-processador.
- Um exemplo de uso dessa diretiva é: #include <stdio.h>.

A diretiva #include adiciona o arquivo .h (sendo h a abreviação de header, que significa “cabeçalho”), em que há uma série de funções de entrada e saída de dados, por exemplo:

- `printf()`: Exibe mensagens ou valores na tela (saída padrão).
- `scanf()`: Lê dados digitados pelo usuário (entrada padrão).
- Para comentários longos, utilizamos `/*` no início e `*/` no final. Todo texto delimitado entre esses dois símbolos será ignorado.
- Para comentários curtos em apenas uma linha, usamos `//`.

Estrutura de um Programa em C

A estrutura básica de um programa em C segue um padrão simples:

- Todo programa em C precisa ter uma função especial chamada main(), pois é por ela que o computador começa a executar o código.
- As chaves **{ }** servem para delimitar um bloco de instruções, ou seja, início e fim do bloco.
- Importante destacar também que C é **sensível a maiúsculas e minúsculas**, ou seja, as letras maiúsculas diferem das minúsculas (*case sensitive*).
- A variável Aluno é diferente de aluno e diferente de ALUNO.

As variáveis devem ser declaradas antes de serem usadas, logo no início do bloco.

Estrutura de um Programa em C

Exemplos de programas em linguagem C:

```
#include <stdio.h>

int main() {
    printf("Hello World!\n");
    return 0;
}
```

Explicação:

- `#include <stdio.h>`: inclui o arquivo de cabeçalho da biblioteca padrão de entrada e saída.
- `int main()`: função principal onde o programa começa.
- `{ }`: bloco de código.
- `printf("Hello, World!\n");`: imprime a mensagem na tela.
- `\n`: pula para a próxima linha.
- `return 0;`: indica que o programa terminou corretamente.
- Atenção: na linguagem C, a maioria dos comandos termina com um ponto e vírgula (;).

Interatividade

Qual das afirmações abaixo está correta em relação à linguagem de programação C?

- a) O bloco da função `main()` é iniciado com `[` e finalizado com `]`.
- b) A linguagem C não diferencia letras maiúsculas e minúsculas em nomes de variáveis.
- c) Comentários em C começam com `*` e terminam com `*`.
- d) A diretiva `#include` insere um arquivo `.h` associado a uma biblioteca.
- e) Não é permitido declarar variáveis na linguagem C.

Resposta

Qual das afirmações abaixo está correta em relação à linguagem de programação C?

- a) O bloco da função main() é iniciado com [e finalizado com].
- b) A linguagem C não diferencia letras maiúsculas e minúsculas em nomes de variáveis.
- c) Comentários em C começam com * e terminam com *.
- d) A diretiva #include insere um arquivo .h associado a uma biblioteca.
- e) Não é permitido declarar variáveis na linguagem C.

Tipos de Dados Básicos em C

Tipos de Dados em C (Tipos de Dados Primitivos):

Tipo	Tamanho	Exemplo	Descrição
char	1 byte	'a'	Caractere único
int	4 bytes (geralmente)	10	Inteiro
float	4 bytes	8.55	Decimal simples
double	8 bytes	3.141592	Decimal com mais precisão

- Exemplo: declarar e atribuir um caractere a uma variável do tipo char.

```
char letra = 'A';
```

- char: Armazena um único caractere, como letras, dígitos ou símbolos.
- Imprimindo o caractere armazenado na variável letra.

```
printf("O caractere armazenado é: %c", letra);
```


Tipos de Dados Básicos em C

O tipo `char` pode ser utilizado na função `scanf("%c",&variavel);` para capturar um caractere do teclado. Exemplo de entrada e saída de um `char`, armazenado na variável *letra*:

```
char letra;
```

```
printf("Digite um caractere: ");
```

```
scanf("%c", &letra);
```

```
printf("Voce digitou: %c\n", letra);
```

- O `%c` é utilizado no `scanf()` para receber um caractere (`char`) do usuário. No `printf()`, ele é usado para exibir caracteres individuais na tela.
- A variável *letra* armazena um único caractere digitado pelo usuário.

Tipos de Dados Básicos em C

Lendo uma string (vetor de char): Para armazenar e ler uma string, podemos usar: `scanf("%s", nome);` nome é um vetor de char, declarado, por exemplo, como:

```
char nome[20]; //suporta até 19 caracteres + '\0'
```

```
printf("Digite seu nome: ");
```

```
scanf("%19s", nome); // evita estouro de buffer
```

```
printf("Seu nome é: %s\n", nome);
```

O `scanf("%s", nome);` não lê espaços. Para ler espaços com `scanf`, utilize `scanf("%[^\n]", nome);`. Exemplo:

```
scanf("%19[^\n]", nome);
```

Tipos de Dados Básicos em C

- Tipo int (inteiro): utilizado para armazenar números inteiros, positivos ou negativos, sem casas decimais.

Exemplo:

```
int num1 = 5, num2 = 10; //Declaração e inicialização das variáveis
```

Podemos ler variáveis do tipo inteiro:

```
int num1, num2; // Declaração das variáveis

// Solicitando entrada do usuário
printf("Digite dois números inteiros: ");
scanf("%d %d", &num1, &num2); // Lendo os valores
```

Podemos imprimir os valores das variáveis do tipo inteiro:

```
// Exibindo os valores lidos
printf("O primeiro número digitado foi: %d\n", num1);
printf("O segundo número digitado foi: %d\n", num2);
```

Tipos de Dados Básicos em C

Tipos reais:

- **float** (ponto flutuante (decimal)): Usado para armazenar números com casas decimais, ou seja, valores fracionários.
- **double** (ponto flutuante de dupla precisão): Semelhante ao float, mas com maior capacidade e precisão (cerca de 15 dígitos significativos).

O separador decimal, no caso da linguagem C, é o ponto (.), e não a vírgula. Exemplo:

```
// Declarações com tipo float
float notaAluno = 0;
float valor1 = 5.0, valor2 = 8.8;
float cotacaoDolar = 5.7591;

// Declarações com tipo double
double raio, pi = 3.14159;
```

Tipos de Dados Básicos em C

- Tipos reais: leitura e impressão dos dados.
- No scanf: é obrigatório diferenciar: %f → para float %lf → para double.

```
float a;  
double b;
```

```
scanf ("%f", &a);  
scanf ("%lf", &b);
```

- No printf, tanto para float quanto para double, o especificador é o mesmo: %f. Isso acontece porque, por padrão no printf, os valores float são promovidos a double.

```
float x = 5.0f;  
double y = 8.8;
```

```
printf ("%f", x);  
printf ("%f", y);
```

Interatividade

Analise as afirmações sobre tipos de dados na linguagem C:

- I. int – utilizado para declarar variáveis que armazenam números inteiros, positivos ou negativos.
- II. float – utilizado para declarar variáveis que armazenam números reais com casas decimais e precisão simples.
- III. double – assim como o int, representa apenas números inteiros e positivos.
- IV. char – utilizado para armazenar um único caractere, como uma letra ou símbolo.

Está correto o que se afirma em:

- a) I, II e III, apenas.
- b) I, II e IV, apenas.
- c) I, II, III e IV.
- d) I, apenas.
- e) Nenhuma das alternativas anteriores.

Resposta

Analise as afirmações sobre tipos de dados na linguagem C:

- I. int – utilizado para declarar variáveis que armazenam números inteiros, positivos ou negativos.
- II. float – utilizado para declarar variáveis que armazenam números reais com casas decimais e precisão simples.
- III. double – assim como o int, representa apenas números inteiros e positivos.
- IV. char – utilizado para armazenar um único caractere, como uma letra ou símbolo.

Está correto o que se afirma em:

- a) I, II e III, apenas.
- b) I, II e IV, apenas.
- c) I, II, III e IV.
- d) I, apenas.
- e) Nenhuma das alternativas anteriores.

Identificadores e Variáveis

- Identificadores: Em C, os nomes atribuídos a variáveis, funções e outros elementos são chamados de identificadores.

Regras básicas:

- Um identificador não pode ter o mesmo nome que uma palavra-chave reservada da linguagem C, como int, return, if, entre outras, nem das funções.
- O primeiro caractere deve ser uma letra (a-z ou A-Z) ou um sublinhado (_). Os caracteres subsequentes podem ser letras, números (0-9) ou sublinhados.
 - Um identificador não pode conter espaços, caracteres especiais (como @, #, !) ou começar com números.
 - Identificadores diferenciam letras maiúsculas e minúsculas (case-sensitive).

Identificadores e Variáveis

Exemplos de identificadores válidos:

```
int contador;  
float _media;  
char nomeCliente;  
int teste123;
```

Esses exemplos são válidos na linguagem C, pois seguem as regras de nomenclatura para identificadores:

- *contador*; começa com uma letra minúscula.
- *_media*; começa com um sublinhado (permitido em C).
- *nomeCliente*; usa letras minúsculas e maiúsculas no estilo camelCase.
- *teste123*; contém números após o início, o que é permitido.

Identificadores e Variáveis

Exemplos de identificadores inválidos:

```
int 1numero;  
float valor$;  
char nome cliente;
```

Esses exemplos são inválidos em C porque:

- 1numero começa com número, identificadores devem começar com letra ou sublinhado.
- valor\$ contém o caractere especial \$, apenas letras, números e _ são permitidos.
- nome cliente contém espaço, identificadores não podem conter espaços.

Identificadores e Variáveis

- Variáveis: uma variável é uma posição nomeada de memória, que é usada para guardar um valor que pode ser modificado pelo programa.
- Em C, todas as variáveis precisam ser declaradas antes de serem utilizadas.

A estrutura básica de uma declaração em C segue o formato:

tipo lista_de_variáveis;

- *tipo* representa qualquer tipo de dado válido da linguagem.
- *lista_de_variáveis* corresponde a um ou mais identificadores.

Exemplo:

```
int i, j;
```

```
float nota = 10.0;
```

tipo

identificador

valor inicial (inicialização)

Identificadores e Variáveis

- Escopo de variáveis: As variáveis estão associadas a um escopo, podendo ser declaradas dentro dele e tendo sua visibilidade limitada a esse escopo.
- Na linguagem C, utilizamos delimitadores de escopo para indicar onde um bloco de código começa e termina. Eles definem o contexto em que variáveis, funções e instruções são visíveis e executadas.
- Chaves { }: São os principais delimitadores de escopo em C. Tudo que está entre { e } pertence a um mesmo bloco de código.
- É possível criar escopos aninhados (blocos dentro de blocos).

Por padrão, elas podem ser declaradas em três lugares básicos:

- Dentro de funções: estas são variáveis locais.
- Na definição dos parâmetros das funções: parâmetros formais.
- Fora de todas as funções: variáveis globais, respectivamente.

Identificadores e Variáveis

- Variáveis Globais: As variáveis globais são aquelas declaradas fora de qualquer função, geralmente no início do programa, antes da função main().

Características:

- Escopo: Podem ser acessadas e modificadas por qualquer função dentro do programa.
- Duração: Existem durante toda a execução do programa, sendo criadas no início e destruídas ao final.
- Memória: Usam a área global ou área de dados na memória.

Identificadores e Variáveis

- Variáveis Locais: As variáveis locais são aquelas que são declaradas dentro de funções ou blocos de código, delimitados por chaves { }. Seu escopo é restrito ao bloco onde foram definidas, o que significa que só podem ser acessadas dentro desse mesmo trecho de código.

Características:

- Escopo: Só podem ser acessadas dentro da função ou bloco onde foram declaradas.
- Duração: São criadas quando a função/bloco é executada e destruída quando a execução termina.
 - Memória: Usam a pilha (stack), garantindo que cada função tenha suas próprias variáveis locais.

Interatividade

Analise as afirmações sobre as funções `printf()` e `scanf()` na linguagem C:

- I. O comando `printf` é uma função da linguagem C utilizada para exibir informações na tela.
- II. No `printf` o caractere de escape `\n` provoca uma quebra de linha, movendo o cursor para a próxima linha no console.
- III. Sempre use o operador `&` antes do nome da variável, exceto para strings (arrays de caracteres), pois o nome do array já representa o endereço de memória.
- IV. O `scanf()` pode ser problemático se a entrada do usuário não corresponder ao formatador especificado.

Está correto o que se afirma em:

- a) I, II e III, apenas.
- b) I, II e IV, apenas.
- c) I, II, III e IV.
- d) I, apenas.
- e) Nenhuma das alternativas anteriores.

Resposta

Analise as afirmações sobre as funções printf() e scanf() na linguagem C:

- I. O comando printf é uma função da linguagem C utilizada para exibir informações na tela.
- II. No printf o caractere de escape \n provoca uma quebra de linha, movendo o cursor para a próxima linha no console.
- III. Sempre use o operador & antes do nome da variável, exceto para strings (arrays de caracteres), pois o nome do array já representa o endereço de memória.
- IV. O scanf() pode ser problemático se a entrada do usuário não corresponder ao formatador especificado.

Está correto o que se afirma em:

- a) I, II e III, apenas.
- b) I, II e IV, apenas.
- c) I, II, III e IV.
- d) I, apenas.
- e) Nenhuma das alternativas anteriores.

Operadores Básicos em C

- Os operadores básicos em C são essenciais para a manipulação de dados e controle de fluxos.

Os operadores em C podem ser classificados em diferentes categorias:

- Operadores de Atribuição.
- Operadores Aritméticos.
- Operadores Relacionais.
- Operadores Lógicos.
- Operadores de Incremento/Decremento.

Operadores Básicos em C

Operadores de atribuição são usados para atribuir valores a variáveis. O operador básico é o **=**, que atribui o valor à direita à variável à esquerda. Além disso, existem operadores combinados. Seguem alguns exemplos:

<u>Operador</u>	<u>Equivalente</u>	<u>Exemplo</u>	<u>Resultado</u>
=	$x = y$	$x = y$	Atribui y a x
+=	$x = x + y$	$x += y$	Soma y a x e armazena em x
-=	$x = x - y$	$x -= y$	Subtrai y de x e armazena em x
*=	$x = x * y$	$x *= y$	Multiplica x por y e armazena em x

Operadores Básicos em C

- Operadores aritméticos são utilizados para realizar operações matemáticas básicas, como soma (+), subtração (-), multiplicação (*), divisão (/) e o operador de módulo (%) que retorna o resto de uma divisão entre dois inteiros.

Exemplo:

```
#include <stdio.h>
#include <locale.h>

int main() {
    setlocale(LC_ALL, "Portuguese");

    int a = 10, b = 3;

    printf("Soma: %d\n", a + b);
    printf("Subtração: %d\n", a - b);
    printf("Multiplicação: %d\n", a * b);
    printf("Divisão: %d\n", a / b);
    printf("Resto: %d\n", a % b);

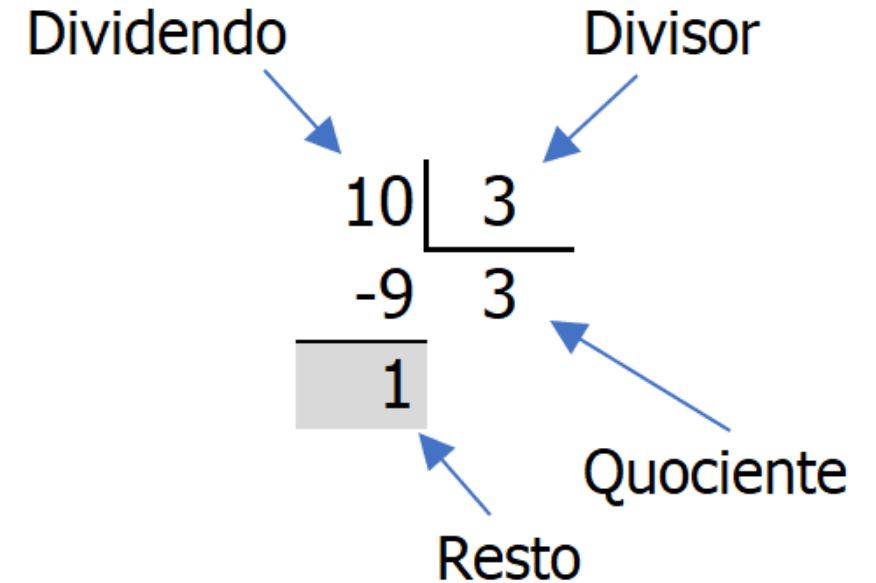
    return 0;
}
```

Operadores Básicos em C

- Operador de módulo (%) Resto da Divisão: O operador % retorna o resto da divisão inteira.

Exemplos desta operação: Resto da Divisão ($a \% b$):

- $10 \% 3 = 1$ (resto da divisão de 10 por 3).
- No caso, $10 \% 3$ resulta em 1, porque 10 dividido por 3 é 3 com resto 1.



- $10 \% 2$ retorna 0, porque 10 dividido por 2 é 5 com resto 0.
- $9 \% 3$ retorna 0, porque 9 dividido por 3 é 3 com resto 0.
- $9 \% 2$ retorna 1, porque 9 dividido por 2 é 4 com resto 1.

Operadores Básicos em C

- Operadores Relacionais: utilizamos os operadores relacionais para comparar dois valores, sejam eles variáveis ou constantes.
- Exemplo de operadores relacionais: (==, >, <, >=, <=, !=).

Operador	Descrição	Exemplo	Resultado
==	Igual a	5 == 5	1 (true)
>	Maior que	10 > 5	1 (true)
<	Menor que	5 < 10	1 (true)
>=	Maior ou igual a	10 >= 11	0 (false)
<=	Menor ou igual a	5 <= 10	1 (true)
!=	Diferente de	5 != 5	0 (false)

Operadores Básicos em C

- Operadores Lógicos: operadores lógicos permitem a construção de expressões lógicas. Eles podem combinar múltiplas condições dentro de uma instrução condicional.

Em C, existem três operadores lógicos principais:

<u>Operador</u>	<u>Descrição</u>
AND (&&)	Retorna verdadeiro (1) apenas se ambas as condições forem verdadeiras.
OR ()	Retorna verdadeiro (1) se pelo menos uma das condições for verdadeira.
NOT (!)	Inverte o valor lógico de uma condição, transformando verdadeiro em falso e vice-versa.

Ordem de precedência dos operadores lógicos:

!	(NOT)	- Mais alto
&&	(AND)	
	(OR)	- Mais baixo

- Se houver múltiplos operadores em uma expressão sem parênteses, o operador de maior precedência será avaliado primeiro.

Operadores Básicos em C

- Operadores de Incremento/Decremento: são usados para aumentar ou diminuir o valor de uma variável.
- Podemos utilizar para somar ou subtrair 1 de uma variável.

++ Incrementa em 1 (x++ ou ++x)

-- Decrementa em 1 (x-- ou --x)

Exemplo:

```
int contador = 0;  
contador++;  
printf("Contador: %d\n", contador); // Imprime Contador: 1
```

Interatividade

Analise o código abaixo escrito em C. Marque a alternativa que apresenta o valor de saída na impressão deste programa.

a) 5 5 5 5.

b) 6 6 6 6.

c) 6 5 5 5.

d) 5 6 5 6.

e) 5 5 6 6.

```
#include <stdio.h>

int main() {
    int a = 5, b = 5;

    printf("%d ", a);
    printf("%d ", ++a);
    printf("%d ", b++);
    printf("%d.", b);

    return 0;
}
```


Resposta

Analise o código abaixo escrito em C. Marque a alternativa que apresenta o valor de saída na impressão deste programa.

```
#include <stdio.h>

int main() {
    int a = 5, b = 5;

    printf("%d ", a);
    printf("%d ", ++a);
    printf("%d ", b++);
    printf("%d.", b);

    return 0;
}
```

a) 5 5 5 5.

b) 6 6 6 6.

c) 6 5 5 5.

d) 5 6 5 6.

e) 5 5 6 6.

Referências

- BACKES, André. *Linguagem C: completa e descomplicada*. São Paulo: Grupo GEN, 2018.
- CELES, Waldemar; RANGEL, Renato; EIRAS, José. *Introdução a estruturas de dados: com técnicas de programação em C*. Rio de Janeiro: Elsevier/Grupo GEN, 2016.
- DAMAS, Luiz. *Linguagem C*. Rio de Janeiro: LTC, 2011.
- DEITEL, Paul; DEITEL, Harvey. *C: como programar*. 7. ed. São Paulo: Pearson Prentice Hall, 2013.
- MANZANO, José Augusto N. G.; OLIVEIRA, Jayr Figueiredo de. *Algoritmos: lógica para desenvolvimento de programação de computadores*. São Paulo: Editora Saraiva, 2016.

ATÉ A PRÓXIMA!